The Two Faces of Abstraction Regret: Control-Flow and Memory-Layout Limits of GPU DSLs on Irregular Automata

Alessandro Potenza
Politecnico di Milano
Milan, Italy
alessandro1.potenza@mail.polimi.it

***Abstract*—GPU domain-specific languages (DSLs) such as OpenAI Triton deliver near-CUDA performance at far lower effort on *regular* tensor algebra. We ask what that abstraction costs on *irregular* workloads and answer it with a metric we call *abstraction regret*: the performance a DSL forecloses—with the algorithm held fixed—because it cannot express the memory layout or control flow a workload needs. We decompose regret along these two capability axes and instantiate it on finite automata across the paradigm axis CUDA and NVIDIA Warp (thread-SIMT) versus Triton and its low-level Gluon frontend (tile-SPMD). Automata expose two complementary faces: an NFA active-set traversal that is *control-flow bound*, and a DFA dense-table walk that is *memory bound* (its throughput halves as the table crosses L2). On both faces the regret is large for the tile-SPMD DSLs and small for the thread-SIMT ones—Triton pays 5–$13\times$ versus CUDA across the two faces, while Warp, an equally high-level *Python* DSL, matches or beats hand CUDA on the NFA (0.6–$0.9\times$) and pays only 1.4–$2.3\times$ on the DFA. So regret is set by the execution *paradigm*, not by how high-level the DSL looks. We make the attribution falsifiable with the Triton$\leftrightarrow$Gluon controlled pair (identical MLIR compiler stack; Gluon only adds explicit layout/shared-memory control): Gluon *still* cannot express the kernel, so the binding constraint is the paradigm, not tuning or layout. A two-parameter cost model corroborates the regret (predictive for the thread model; holdout 2.7%) and we name the missing IR primitives (scalar gather in a tile, register-resident bitset, data-dependent loop). Along the way we build a portable work-efficient automata engine ($\approx 330\times$–$10^4\times$ over a faithful full scan, 15–170 Gbps, validated bit-for-bit against a CPU oracle on six real ANMLZoo automata up to 48k states). We confirm the centerpiece on a second GPU architecture (NVIDIA A100): the 2×2 regret pattern and the architecture-independent tile-SPMD scalar ceiling both reproduce.**

I. Introduction

GPU domain-specific languages such as OpenAI Triton [16] have moved from research to the mainstream—Triton is the default code-generation backend for PyTorch—because they deliver near-CUDA performance on dense tensor algebra at a fraction of the engineering effort. That productivity comes from *hiding* the very things a GPU programmer otherwise manages by hand: thread indexing, memory placement, and control flow. On *regular* workloads—dense matrix and tensor kernels with static, coalesced access and uniform control—this hiding is essentially free, which is precisely why these DSLs succeed.

Irregular workloads are the opposite regime. Graph traversal, sparse algebra, and finite automata are dominated by data layout and data-dependent control flow rather than arithmetic [28], and decades of evidence show that the *representation*, at a fixed algorithm, swings their GPU performance by an order of magnitude [27], [26]. What, then, does a GPU DSL's hiding *cost* on irregular workloads—and what, exactly, causes that cost? The question is open: DSL benchmarks are dense-tensor-only, and the GPU automata engines that define the state of the art—iNFAnt [1], ngAP [3], HybridSA [4], BitGen [5], ANG [17]—are uniformly single-stack CUDA, framed as faster *algorithms*, never as what a programming abstraction forecloses. No prior work compares modern GPU kernel DSLs on an irregular workload.

We answer the question on finite-automata simulation, a canonical irregular workload (deep-packet inspection, regular-expression matching, bioinformatics) with *two complementary faces*: an NFA active-set traversal that is control-flow-bound, and a DFA dense-table walk that is memory-bound. We name the cost *abstraction regret*—the performance a DSL forecloses, with the algorithm held fixed, because it cannot express the memory layout or control flow a workload needs—and find it is large (Triton pays 6–$8\times$ versus CUDA). The central, and initially counter-intuitive, result is *what causes it*: not how high-level the DSL is, but its *execution paradigm*. A clean 2×2 (abstraction-height $\times$ paradigm) factorial over four real DSLs shows the thread-SIMT column (low-level CUDA and *high-level* Warp) sits at $\approx 1\times$ while the tile-SPMD column (high-level Triton and *low-level* Gluon) fails. We make the attribution causal—a controlled same-MLIR-stack Triton$\leftrightarrow$Gluon pair, plus a $16\times$ tile-vs-scalar cliff measured *within* one DSL—and actionable: both faces reduce to a *single* missing IR primitive, a scalar gather within a tile.

Contributions. (A) *Abstraction regret, operationalized and falsified to a cause*: we show the regret is set by the *execution paradigm* (tile-SPMD vs thread-SIMT), *not* abstraction height, via a clean 2×2 (height $\times$ paradigm) factorial whose four cells are real DSLs (CUDA, Warp, Triton, Gluon). The attribution is made *falsifiable*, not rhetorical, by a controlled Triton↔Gluon pair (same MLIR stack, Gluon only *adds* layout control yet *still* cannot express the kernel)—which also defeats the autotuning counter-thesis [34], since no knob exists on the lower-level sibling. We decompose regret along two capability axes (control-flow vs memory-layout) on the *two faces* of automata (control-flow-bound NFA, memory-bound DFA), back it with a holdout-validated cost model (§III), and distill an *actionable* capability→cost map naming the missing IR primitive (scalar-gather-in-tile) and its price— a result for GPU-DSL designers, distinct from hardware performance portability [9]. **(B)** *A work-efficient portable NFA engine*: an active-set/worklist bit-packed kernel (§IV) that removes the $O(n^2)$ compute wall and reaches 15–170 Gbps. **(C)** *A reproducible artifact*: one registry-based framework, a CPU reference oracle, median+CI95 sweeps, and figures regenerated only from versioned CSVs.

II. Background

Automata, and their two faces. An *NFA* maintains a set of active states; per input symbol it applies the transition relation (each active state's out-edges labelled by that symbol) and then an epsilon-closure (following label-free edges to fixpoint). GPU engines since iNFAnt [1] store the transitions in CSR form—symbol- and epsilon-edge arrays indexed by per-state row pointers—and represent the active set as a bit-vector. The work per symbol is data-dependent (which states are active, how many out-edges each has) and scatter-heavy: this is the *control-flow-bound* face. A *DFA* is the deterministic dual: a single dense transition table $T[\textit{state}, \textit{symbol}]$ ($n \times 256$) with no active set, walked by one random gather $T[s \cdot 256 + c]$ per input byte. For large n the table exceeds cache, so the DFA is the *memory-bound* face. The two faces stress disjoint hardware limits, which is why we study both. We use the standard ANMLZoo [7]/AutomataZoo benchmark families; Hyperscan [6] is the CPU baseline.

GPU automata engines from iNFAnt to AsyncAP [2], ngAP [3] (non-blocking + memoization + privatization), HybridSA [4] (bit-parallel), BitGen [5] (Parabix bitstreams) and ANG [17] win by reorganizing or reducing irregular memory traffic—each a new *algorithm*, hand-written in CUDA, that bundles the memory effects in. None isolates what a programming *abstraction* costs.

Two GPU execution paradigms (the axis we study). A *thread-SIMT* DSL (CUDA, and NVIDIA Warp) is written from one thread's point of view: scalars, data-dependent branches, per-thread loops, bit manipulation and a register-resident working set are all native, so the NFA's per-state traversal maps directly. A *tile-SPMD* DSL (Triton, and its low-level frontend Gluon) operates on whole *tiles* (blocks) of data, with the compiler owning the intra-tile layout and thread mapping; this is ideal for dense, uniform tensor algebra but a per-element scalar gather or a data-dependent inner loop must be emulated, not expressed. Crucially, *abstraction height* (how much the DSL hides) is *orthogonal* to the paradigm: Warp is high-level yet thread-SIMT, while Gluon is low-level yet tile-SPMD. Our 2×2 study exploits exactly this orthogonality to separate the two candidate causes of the regret.

The gap. Every GPU automata engine above is CUDA-only, and **no prior work compares modern GPU kernel DSLs (Triton/Gluon vs CUDA vs Warp) on an irregular workload**: published GPU-DSL benchmarks target dense tensor kernels, and no irregular-workload suite exercises these kernel DSLs. We close that gap and frame the resulting difference as an expressibility cost.

III. Abstraction regret and the cost model

We model per-symbol time as a roofline-style sum $t_{\text{sym}} = \text{traffic}/\text{bandwidth} + n^2 c_2$, where the compute term is *quadratic* because the faithful kernel runs an $O(n)$ transition scan and an $O(n^2)$ epsilon-closure per symbol, and c_2 is fitted per backend; its DSL-to-DSL ratio, at fixed algorithm, corroborates the regret. A holdout test (fit $n \leq 128$, predict $n{=}256$) shows the model is predictive for the thread-model backend (CUDA 2.7% error) but *not* for tile/SPMD Triton (45% error: its large fixed launch overhead at small n is misattributed to the n^2 term). We therefore take the directly-measured throughput ratio as the primary regret metric (Triton 6–8×, robust) and the fitted c_2 ratio (10.1×) as corroborating only.

IV. Implementation

A single registry maps a backend/technique to an executor. The NFA is stored once in CSR and consumed identically by every backend, so comparisons are apples-to-apples. Correctness is gated against a CPU reference oracle (latch-first-match) and its bit-packed executable spec. **CUDA**: `dense` (int8/state), `bitpacked` (register-resident 64-bit words), `multistream` (one thread/string), `multistream_shared` (CSR staged in shared memory—a layout Triton cannot express), `multistream_async` (pinned + streamed overlap), and `worklist` (active-bit iteration via `__ffsll` + frontier eps-closure; $O(\text{active})$, no $O(n^2)$), `worklist_global` (global working set, dynamic word count, no 512-state cap — scales to ANMLZoo-sized automata; register residency costs ∼4–5× vs the capped kernel), and `worklist_warp` (*block-parallel*: one warp per string, the 32 lanes partition the state-words and scatter transitions via `atomicOr`—it spreads one string's loads across the warp, 3–9× faster than the single-thread global kernel on real ANMLZoo automata at a GPU-saturating batch, §VI), and `worklist_shared` (same scheme with the working set staged in dynamic *shared* memory, ≤1536 states—the working-set-layout ablation of the work-efficient kernel). **Triton**: `dense`, `bitpacked`, `multistream`, `worklist` (≤ 64 states, via `libdevice.ffs` + a data-dependent `while` loop). **Warp**: `multistream` (thread-SIMT Python). **Gluon** (Triton's experimental low-level frontend): `gl.load` always returns a layout-typed tensor (no scalar load), so the data-dependent CSR inner loop is inexpressible—a runnable probe (`scripts/gluon_probe.py`) reproduces the compiler error (*"Value argument cannot be block type"*) and exits nonzero if a future Gluon ever compiles it, making the claim falsifiable. **DFA (the memory-bound face):** a `dfa` kernel walks a dense *states* $\times$ 256 table, one random lookup per byte, implemented identically in CUDA, Triton and Warp (one program/thread per string); all match the DFA oracle.

V. Methodology

Fairness protocol (the crux of a DSL comparison). Every backend consumes the *same* CSR automaton representation and implements the *same* algorithm, with kernels structurally mirrored line-for-line from one specification; across a comparison we vary *only* the DSL (and, within a DSL, the technique), never the algorithm. This is what makes the measured gap an *abstraction* cost rather than an algorithmic one, and it is enforced mechanically: a single registry maps a (backend, technique) pair to an executor, and a CPU reference oracle gates correctness.

Correctness oracle. The oracle (`reference.py`) is the NFA/DFA semantics with *latch-first-match* (report at the first accepting configuration). Every backend/technique is verified bit-identical to it—`(accepted, match_len)` on all examples, on randomized fuzz/stress NFAs (up to 500 states), and on six real ANMLZoo automata (below)—so a faster kernel can never be a wrong kernel.

Timing and statistics. Throughput is total input bits over the per-run batch-kernel time on a fixed batch (2048×256 B), reported as median + percentile-bootstrap 95% CI over 9 runs after warmup, with kernel and transfer time separated; GPU timings are non-Gaussian, so we use medians and bootstrap CIs throughout [8]. The headline regret ratios are additionally measured over multiple random-NFA seeds (median + min–max spread) to rule out single-instance artifacts, and parallel-scaling comparisons use a GPU-saturating batch so a baseline is never starved of parallelism.

Bound diagnosis. We classify kernels compute- vs memory-bound with the *instruction* roofline [32], [33]—appropriate for these integer/transaction-dominated FSM kernels, which a FLOP roofline would understate—and corroborate with Nsight Compute counters (SM%, DRAM%, L2 hit, achieved occupancy).

Environment. Hardware and library versions are captured per CSV row. Hardware: NVIDIA RTX 4070 (sm_89, 46 SMs, 12 GB, 6 MB L2); software: CUDA toolkit 13.x, Triton 3.5.1, Warp 1.14, PyTorch 2.9 (cu128). We additionally validate on *six real* ANMLZoo automata spanning six families: Levenshtein (2787 states), Hamming (11349, 2.1M transitions), Brill (42661, 4.4M), Fermi (40786, particle-physics regexes), RandomForest (33223, 6.27M transitions—an ML decision-forest, our densest), and CoreRings (48005, synthetic ring—our largest state count); all loaded from pinned public ANML with correct `all-input`/`start-of-data` semantics. `worklist_global` matches the reference oracle bit-for-bit on every input, confirming correctness at production scale (up to 48k states, 6.3M transitions).

TABLE I

Throughput (Gbps) by technique and NFA size (batched, RTX 4070). Dashes: > 64 states unsupported by the single-word Triton/Warp kernels.

states	CUDA worklist	CUDA full-scan	Triton full-scan	Triton worklist
32	170.2	0.513	0.071	24.4
64	163.8	0.131	0.021	25.2
128	47.6	0.023	0.003	–
256	31.7	0.006	0.001	–
500	15.5	0.0015	0.0002	–

VI. Results

Table I reports throughput.

The faithful kernel is compute-bound; memory layout is irrelevant there. `multistream`, `multistream_shared` (modeled CSR traffic $= 0$), and `multistream_async` tie to within the bootstrap CI at every size, and throughput scales as $1/n^2$. Nsight Compute counters confirm this at the hardware level: the full-scan kernel sustains $\approx$19% of peak SM throughput but only **0.01%** of peak DRAM throughput, and `multistream_shared` shows identical SM%, DRAM% and occupancy—only the L2 hit rate rises (79→93%), without moving runtime. The memory axes cannot help until the algorithm is work-efficient.

The work-efficient kernel unlocks the regime. The `worklist` kernel is $\approx 330\times$–$10^4\times$ faster than full-scan (the speedup grows with n: 332× at 32 states, $\approx 10^4\times$ at 500) and reaches 15–170 Gbps, moving the workload toward memory-bound where the memory techniques become load-bearing.

Abstraction regret is the execution paradigm, not abstraction height. On the same full-scan kernel the directly-measured throughput ratio vs CUDA is Triton 6–8× and Warp $0.9\times$ (Warp is *faster*; Fig. 1); the cost-model fit corroborates (c_2 ratio Triton $10.1\times$, Warp $0.63\times$). The ratios are stable, not single-seed artifacts: across 5 random NFAs $\times$ 3 sizes the Triton regret is median $7.1\times$ (6.6–9.4) and Warp median $0.85\times$ (Warp beats CUDA in the median; `paper/data/regret_multiseed_rtx4070.csv`). The four DSLs form a clean 2×2 factorial (Table II) that separates the two candidate causes. Regret tracks the *column* (execution paradigm), not the *row* (abstraction height): the thread-SIMT column is $\approx 1\times$ at *both* heights (low-level CUDA and high-level Warp), while the tile-SPMD column fails at *both* heights (high-level Triton 6–8×; low-level Gluon cannot express the kernel *at all*). This falsifies the obvious alternative hypothesis "high-level ⇒ slow," and—because Gluon is the lower-level sibling on the *same* MLIR stack—it also rules out tuning/maturity: the gap persists where no knob exists.

Causal control: the regret is caused by the scalar/control-flow primitive. To show the attribution is causal, not correlational, we ablate the primitive *within Triton*, holding language, data, harness and parallelism fixed: one program per string processes the same bytes either (i) as a tile-vectorized reduction or (ii) with a carried data-dependent scalar recurrence (the automata pattern). At a GPU-saturating

TABLE II
THE REGRET IS THE EXECUTION PARADIGM, NOT ABSTRACTION HEIGHT. CELLS ARE DSL (NFA REGRET VS CUDA). REGRET TRACKS THE COLUMN, NOT THE ROW.

	thread-SIMT	**tile-SPMD**
low-level	CUDA (1×)	Gluon (× inexpressible)
high-level	Warp (0.6–0.9×)	Triton (6–8×)

batch the tile kernel scales to 1320 Gbps (Triton's design point) while the scalar kernel hits a hard ≈81 Gbps per-program ceiling—a 16× cliff from the access/control pattern alone (`paper/data/scalar_ablation_rtx4070.csv`). The regret appears *iff* the inexpressible primitive is required. The cross-paradigm half confirms it: thread-SIMT runs the *same* scalar data-dependent recurrence with *no* per-program ceiling—CUDA's DFA walk (a per-byte scalar gather + state recurrence) reaches 163–364 Gbps—whereas tile-SPMD Triton is pinned at a ≈29–81 Gbps scalar ceiling regardless of table size or batch. Same work, same hardware: the ceiling is a property of the paradigm.

Expressibility $\neq$ efficiency. Triton *can* express the work-efficient worklist (unlike Gluon), yet still pays $\approx 6.5\times$ vs CUDA there (CUDA 164 Gbps vs Triton 25 Gbps at ≤ 64 states)—essentially the same regret as the 6–8× on the full scan. Even expressing the right algorithm, the tile/SPMD model imposes a large constant penalty on scalar, data-dependent automata work.

The second face: DFA is memory-bound, and the regret persists. The DFA dense-table walk is the memory-bound dual of the NFA. A fine table-size sweep, median over 3 random DFAs per size (Fig. 2), makes the signature explicit: CUDA throughput rises to a peak *exactly* at the L2 capacity (364 Gbps at the 6 MB table) and then falls 2.2× monotonically to a DRAM-bound plateau (∼163 Gbps) once the table far exceeds L2. Warp tracks the same shape at about half (177 → 108 Gbps). **Triton, by contrast, is flat at** 29–32 **Gbps across the entire 1–100 MB range**—it never enters the memory-bound regime because its tile/SPMD codegen bottlenecks the scalar gather first (DFA regret 5–13×, largest where CUDA peaks at L2). This ≈29 Gbps is a per-program scalar *ceiling*, not a parallelism limit: sweeping batch size, Triton saturates at ≈29 Gbps by ∼4k strings while CUDA keeps scaling past 400 Gbps with the memory idle. So on *both* faces—control-flow-bound (NFA) and memory-bound (DFA)—the regret tracks the execution paradigm, not the workload's bottleneck: it is an intrinsic property of the DSL, not of where the kernel happens to be limited. This is not a single-GPU artifact: on an A100 (40 MB L2, current Triton 3.7) the CUDA knee shifts to ≈32–48 MB (vs ≈6–8 MB on the 4070's 6 MB L2) while Triton stays flat at ≈24–30 Gbps across the whole 2–128 MB sweep—the *same* scalar ceiling as the 4070, three years and 6.7× L2 apart, confirming it is a property of the tile/SPMD model and not the hardware (§IX).

TABLE III
CAPABILITY → COST—AN ACTIONABLE MAP FOR GPU-DSL DESIGNERS: EACH MISSING IR PRIMITIVE AND THE REGRET IT CARRIES. ✓ EXPRESSIBLE, ∼ ONLY AS A STRAINED SINGLE PROGRAM, × INEXPRESSIBLE. REGRET IS THROUGHPUT VS CUDA ON EACH FACE. THE SINGLE PRIMITIVE THAT MOST CLOSES THE TILE-SPMD GAP IS A *scalar gather within a tile*.

Capability (paradigm)	CUDA (thread-SIMT)	Warp (thread-SIMT)	Triton (tile-SPMD)	Gluon (tile-SPMD)
Scalar element load	✓	✓	∼	×
Data-dependent loop	✓	✓	∼	∼
Register-resident bitset	✓	✓	×	×
Explicit shared-mem layout	✓	×	×	✓
NFA regret (control-flow)	1×	0.6–0.9×	6–8×	n/a (×)
DFA regret (memory)	1×	1.4–2.3×	5–13×	—

Capability → cost. Table III maps the capabilities each kernel needs to whether a DSL expresses them and the resulting regret. The thread-SIMT DSLs (CUDA, Warp) express all of scalar load, data-dependent loops, register-resident bitsets and explicit shared-memory layout; the tile-SPMD DSLs cannot express a scalar element load at all (Gluon) or only via a strained single program (Triton), which is exactly what the regret measures. **One root cause, both faces.** Strikingly, both faces reduce to the *same* missing primitive—a *scalar gather within a tile*. The NFA's control-flow regret is the data-dependent per-state scatter/gather; the DFA's memory regret is the per-byte random table gather $trans[s \cdot 256 + c]$. A tile DSL cannot issue either as a scalar element op, so on the DFA Triton never even reaches the memory-bound regime—it is scalar-gather-bound first (hence flat at ≈29 Gbps while CUDA's same gather scales to 364). So the capability→cost map has a single actionable root: give the tile/SPMD IR a first-class scalar gather (and the data-dependent loop it implies), and the regret on *both* faces should collapse—a concrete, falsifiable directive for GPU-DSL designers. This also sharpens the two-axis decomposition into a finding: we examined *both* the control-flow and memory-layout capability axes, and the memory-layout levers turned out *inert* (shared-CSR staging and byte→bit packing tie their global-memory counterparts, §VI)—so on these irregular workloads the binding constraint is control-flow expressibility, even on the memory-bound face. Abstraction regret here is, at root, one axis, not two.

VII. RELATED WORK

Naming. We deliberately invert "abstraction *without* regret" (LMS [12]): staging removes call/dispatch overhead, but cannot manufacture a layout or control-flow pattern the surface abstraction forbids—precisely the residual we measure.

Closest prior on expressibility limits. Two recent works come closest, on orthogonal axes, and neither runs a controlled cross-DSL comparison on an irregular workload. *Hexcute* [13] decomposes the Triton↔CUDA gap into layout-synthesis vs dataflow—but as a *compiler* that *removes* the gap on *dense* GEMM. *STeP* [23] (ASPLOS'26) makes almost our control-flow diagnosis (tile/dataflow abstractions force data-dependent

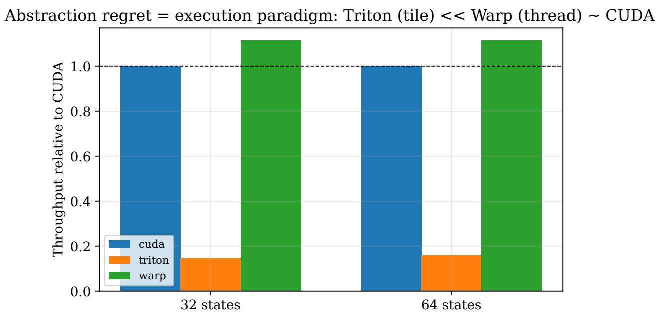

Fig. 1. Throughput relative to CUDA: Triton (tile) $\ll$ Warp (thread) $\approx$ CUDA. Regret is the execution paradigm, not abstraction height.

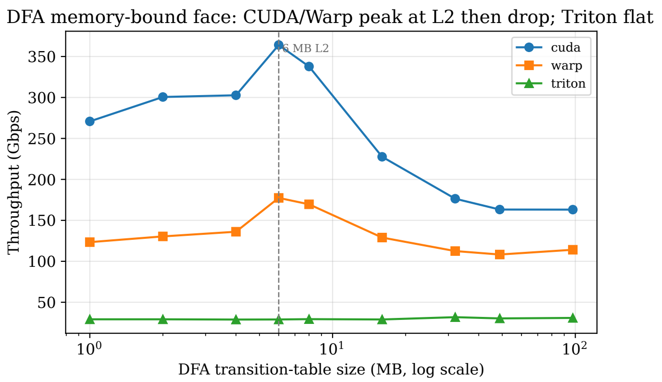

Fig. 2. DFA (memory-bound face), throughput vs table size (log x), median over 3 random DFAs/size: CUDA/Warp peak at the 6 MB L2 then drop $\sim$2.2$\times$ to a DRAM plateau; Triton stays flat (29–32 Gbps)—scalar-gather-bound, never reaching the memory regime.

control flow to be static or unoptimized, and scalar-only primitives block tiling)—but for *spatial dataflow accelerators*, and again ships a solution rather than a measured cross-DSL regret. *GraphIt* [24] is the nearest *DSL-for-an-irregular-workload*, but it *presupposes* the primitives are expressible and autotunes a layout/traversal *schedule*; our point is expressibility itself—of control flow, not just layout—is the gate. *Tawa* [14] and *Descend* [15] argue *qualitatively* that a model's execution paradigm forecloses patterns; we quantify it with a falsifiable Triton$\leftrightarrow$Gluon control and a capability$\rightarrow$cost table. That data-dependent control flow is the hard case for *tiled/SIMD* execution is itself long known [31], supporting that the limit is the *paradigm*, not Triton's maturity. We rebut the autotuning counter-thesis [34] and the LLM kernel-generation line: searching the Triton schedule space cannot manufacture an IR primitive the paradigm lacks—Gluon, the explicit-control sibling on the same MLIR stack [16], still cannot express the kernel.

Abstraction and schedule languages. The Halide [10]/TVM [35] lineage established that abstraction constrains the *schedule* space; Graphene [30] that tensor IRs can be *too inexpressive* for required data-to-thread mappings. These target *dense* tensors; we add the irregular, control-flow face. **Performance portability** [9] decomposes efficiency across a *hardware set*; we decompose across the *expressible-capability* axis at fixed hardware and algorithm—orthogonal, and falsifiable via a named missing primitive rather than a harmonic-mean efficiency.

Irregular GPU workloads. That irregular codes are bound by control divergence and data-dependent memory access, not arithmetic, is the canonical characterization [28]; SpMV format selection [27], [26] and the sparse-tensor compiler [25] show layout/representation alone swings irregular GPU performance by $\sim$10$\times$ at fixed algorithm. Frontier/work-efficient traversal [29], [11] is the method-analogy for our worklist. Automata are the *dynamic-control* member of this family (a small register-resident active set evolving per symbol) that lets us isolate the control-flow face those workloads fix offline (SpMV) or have already abstracted (graph frameworks).

GPU automata engines. The SOTA line—iNFAnt [1], AsyncAP [2], ngAP [3], HybridSA [4], BitGen [5], ANG [17], AutomataBLAS [18] (AP-as-SpMV, our DFA-face comparator)—and AP accelerators [19], plus portable-automata languages [20] and the Parabix bitstream lineage [21], are all single-stack (CUDA/accelerator) and framed as *faster engines*, never as what a programming abstraction forecloses. Each reports a speedup over *its own* baseline/hardware (ngAP 7.9$\times$, ANG 11.7$\times$, BitGen 19.5$\times$—not comparable in absolute Gbps) and is a new *algorithm* on CUDA, whereas our axis is orthogonal. We therefore *position*, not benchmark, against them; matching ngAP/ANG-class absolute throughput is explicit future work (§IX). To our knowledge no prior work studies DSL expressibility across multiple GPU programming models on irregular automata.

VIII. Threats to Validity

Construct. The NFA cross-DSL regret is necessarily measured at $\leq$64 states (the single-word Triton/Warp kernels' cap); two facts guard against a small-n artifact. First, the *DFA* cross-DSL regret is measured at *full scale*—tables up to 100k states / 100 MB, far past L2—and is equally large (5–13$\times$), so the regret is not a small-n effect. Second, most NFA points use random automata, which we back with six real ANMLZoo families (2.8k–48k states, up to 6.3M transitions) where the GPU output matches the oracle bit-for-bit and the block-parallel kernel gives 1.4–27$\times$ over the single-thread baseline (`paper/data/`). *Absolute throughput on these large, sparse-active automata is low (sub-Gbps to a few Gbps)*—the faithful per-string mapping is far from ngAP/ANG-class scheduling; this is the algorithmic gap we explicitly do not close, and it does not affect the cross-DSL *regret* (a relative measure at fixed algorithm). *Internal.* Every backend/technique is checked against a single CPU oracle (latch-first-match) on examples and randomized fuzz/stress NFAs; timings use median + bootstrap CI on a fixed batch with warmup, separating kernel from transfer. *Implementation-effort asymmetry* (the key risk for a DSL comparison): the Triton 5–13$\times$ gap could merely reflect a weaker Triton implementation rather than an inherent limit. Three facts argue against this: (i) the fairness protocol

(§V) holds the algorithm fixed and mirrors kernels line-for-line from one CSR spec; (ii) *Warp*—an equally high-level Python DSL written with comparable effort—matches or beats hand CUDA, so “high-level $\Rightarrow$ slow” is not the explanation; and (iii) the controlled Triton$\leftrightarrow$Gluon pair and the within-Triton $16\times$ tile-vs-scalar cliff show the gap survives where no tuning knob exists—the tile/SPMD execution model is the cause. *External.* Most results are from one GPU (RTX 4070); we re-ran the centerpiece on a second architecture (NVIDIA A100, §IX) and the 2×2 *holds*: the Triton regret rescales (6–$8\times \to 3.2\times$) but stays on the tile/SPMD column, Warp remains $\leq$CUDA ($0.80\times$), and the Triton DFA line stays flat on a *current* Triton 3.7 stack. The cost-model constants are per-backend fits that rescale across architectures, but the *relative* regret—which is the claim—is reproduced cross-arch.

IX. Limitations and future work

Generality across architectures. Most results are from one GPU (RTX 4070, sm_89, 6 MB L2); the qualitative claims are architecture-independent (the capability table is a property of the DSL compilers, and the Triton$\leftrightarrow$Gluon control holds at compile time). We re-ran the centerpiece on a *second* architecture—an NVIDIA A100 (40 MB L2), on a current Triton 3.7 stack—and every load-bearing claim holds. *(i) The 2×2 regret pattern reproduces:* the NFA regret vs CUDA is Triton $3.2\times$ (flat across 32/48/64 states, 3 seeds) and Warp 0.80–$0.83\times$ (`paper/data/regret_a100.csv`). The absolute factors rescale from the 4070 (6–$8\times$ / $0.9\times$)—a bigger, higher-bandwidth GPU narrows the tile/SPMD penalty—but the *structure* is identical: Triton (tile/SPMD) pays regret, Warp (thread-SIMT) matches or beats hand CUDA, so regret tracks the paradigm column, not abstraction height, on both architectures. *(ii) The Triton DFA ceiling is architecture-independent:* Triton throughput stays flat at $\approx$24–30 Gbps across the entire 2–128 MB table sweep—essentially the same flat value as the 4070’s 29–32 Gbps—never reaching the memory-bound regime, while CUDA reaches 73–531 Gbps by exploiting the larger L2 (DFA regret 3–$18\times$; `paper/data/dfa_knee_rich_a100.csv`). That the scalar-gather ceiling is the *same absolute number* on two GPUs three years and $6.7\times$ L2 apart is strong evidence it is a property of the tile/SPMD model, not the hardware. *(iii) The DFA memory-bound knee shifts with L2:* in a cleaner single-backend sweep CUDA falls to its DRAM plateau past $\approx$6–8 MB on the 4070 but stays high through 32–48 MB on the A100—the knee moves by $\approx 6\times$, tracking the $6.7\times$ larger L2 (`paper/data/dfa_knee_a100.csv`). This closes the chief external-validity gap: the contribution is reproduced, not merely re-predicted, on a second architecture. *Kernel.* The single-thread worklist under-utilizes the GPU on large automata (Nsight: 17% occupancy); a block-parallel `worklist_warp` (one warp/string, 32 lanes partitioning the state-words, scattering via `atomicOr`) restores it—3–$9\times$ faster on real ANMLZoo automata at a saturating batch, occupancy $17 \to 57\%$. Two negative results bound the remaining gap: shared-memory working-set privatization only ties the global kernel (0.99–$1.10\times$) and a compacted active-ID worklist barely beats the bitmap scan (0.8–$1.5\times$)—once work-efficient the kernel is *latency-bound*, not memory- or layout-bound (DRAM $\leq$2.25%, L2 hit $\geq$97.6% even for brill’s 17 MB CSR, since all strings share the CSR and only a hot row subset is touched). The remaining gap to ngAP-class throughput is therefore *algorithmic* (cross-string/symbol redundancy: memoization, non-blocking multi-symbol processing), not memory residency or worklist representation—the next step.

X. Conclusion

On irregular automata the GPU-DSL promise breaks along a measurable axis we call *abstraction regret*: the performance a DSL forecloses at a fixed algorithm. Across the two faces of automata—the control-flow-bound NFA and the memory-bound DFA—the regret is large for tile-SPMD DSLs (Triton 6–$8\times$) and absent for thread-SIMT ones (Warp $\leq 1\times$). A 2×2 (height $\times$ paradigm) factorial localizes the cause to the *execution paradigm*, not abstraction height; a controlled same-stack Triton$\leftrightarrow$Gluon pair and a within-Triton $16\times$ tile-vs-scalar cliff make that attribution causal and rule out tuning; and both faces reduce to a *single* missing IR primitive.

The takeaway is constructive and actionable. “Abstraction regret” is not an argument against high-level GPU DSLs—Warp shows a high-level DSL can match hand CUDA on exactly this irregular workload—but a diagnosis with a falsifiable, named cure: **give a tile/SPMD IR a first-class scalar gather within a tile** (and the data-dependent loop it implies), and our map predicts the regret on *both* the control-flow and memory faces should collapse. The method generalizes beyond automata: hold the algorithm fixed, vary the DSL across a controlled pair, and attribute the residual to a named, missing primitive. We release the framework, oracle, kernels, and versioned data so the prediction can be tested as such a primitive lands—and so the same diagnosis can be run on the next irregular workload.

Artifact availability

All code, the CPU reference oracle, the versioned result CSVs, and the figure generator are in the project repository. Every figure and table regenerates from the committed CSVs via a single command (`python paper/figures.py`); the CPU correctness suite runs without a GPU (`pytest -m "not gpu"`), and the GPU kernels and the Gluon falsifiability probe run on any CUDA device. A versioned snapshot with a Zenodo DOI will be minted at release for artifact evaluation. The claim-to-command map is in `docs/REPRODUCIBILITY.md` and a SIGPLAN-style artifact appendix (checklist, install, claims$\to$commands$\to$expected) in `docs/ARTIFACT_APPENDIX.md`.

References

[1] N. Cascarano *et al.*, “iNFAnt: NFA pattern matching on GPGPU devices,” *ACM SIGCOMM CCR*, 2010.

[2] H. Liu, S. Pai, A. Jog, “Asynchronous Automata Processing on GPUs,” *POMACS/SIGMETRICS*, 2023.

[3] T. Ge, T. Zhang, H. Liu, "ngAP: Non-blocking Large-scale Automata Processing on GPUs," *ASPLOS*, 2024.
[4] A. Le Glaunec, L. Kong, K. Mamouras, "HybridSA: GPU Acceleration of Multi-pattern Regex Matching using Bit Parallelism," *OOPSLA*, 2024.
[5] T. Ge, X. Chu, H. Liu, "Interleaved Bitstream Execution for Multi-Pattern Regex Matching on GPUs," *MICRO*, 2025.
[6] X. Wang *et al.*, "Hyperscan: A Fast Multi-pattern Regex Matcher for Modern CPUs," *USENIX NSDI*, 2019.
[7] J. Wadden *et al.*, "ANMLZoo: a benchmark suite for exploring bottlenecks in automata processing," *IISWC*, 2016.
[8] T. Hoefler, R. Belli, "Scientific Benchmarking of Parallel Computing Systems," *SC*, 2015.
[9] S. J. Pennycook, J. D. Sewall, V. W. Lee, "A Metric for Performance Portability," *PMBS@SC / FGCS*, 2016/2019.
[10] J. Ragan-Kelley *et al.*, "Halide: a language and compiler for optimizing parallelism, locality, and recomputation," *PLDI*, 2013.
[11] Y. Wang *et al.*, "Gunrock: A High-Performance Graph Processing Library on the GPU," *PPoPP*, 2016.
[12] T. Rompf, M. Odersky, "Lightweight Modular Staging: Abstraction Without Regret," *CACM* 55(6), 2012.
[13] X. Zhang *et al.*, "Hexcute: A Tile-based Programming Language with Automatic Layout and Task-Mapping Synthesis," arXiv:2504.16214, 2025 (preprint).
[14] Authors, "Tawa: Automatic Warp Specialization for GPU Tile Languages," *CGO*, 2026 (to appear; arXiv:2510.14719).
[15] B. Köpcke, S. Gorlatch, M. Steuwer, "Descend: A Safe GPU Systems Programming Language," *PLDI*, 2024.
[16] P. Tillet, H. T. Kung, D. Cox, "Triton: an intermediate language and compiler for tiled neural network computations," *MAPL@PLDI*, 2019.
[17] Y. Wang *et al.*, "ANG: Accelerating NFA Processing on GPUs via Multi-Level Fine-Grained Parallelism," *ICCD*, 2025.
[18] "Advancing Matrix Operations for Memory-Efficient Automata Processing on GPUs (AutomataBLAS)," *ACM TACO*, 2025.
[19] N. Du, J. Emer, D. Sánchez, "Hopps: Leveraging Sparsity to Accelerate Automata Processing," *ASPLOS*, 2025.
[20] K. Angstadt, W. Weimer, K. Skadron, "RAPID Programming of Pattern-Recognition Processors," *ASPLOS*, 2016.
[21] R. D. Cameron *et al.*, "Bitwise Data Parallelism in Regular Expression Matching," *PACT*, 2014.
[22] J. Wadden *et al.*, "AutomataZoo: A Modern Automata Processing Benchmark Suite," *IISWC*, 2018.
[23] G. Sohn *et al.*, "STeP: A Streaming Abstraction for Dynamic Parallelism on Spatial Dataflow Architectures," *ASPLOS*, 2026 (arXiv:2511.07776).
[24] Y. Zhang *et al.*, "GraphIt: A High-Performance Graph DSL," *OOPSLA*, 2018.
[25] F. Kjolstad *et al.*, "The Tensor Algebra Compiler," *OOPSLA*, 2017.
[26] Z. Ye *et al.*, "SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning," *ASPLOS*, 2023.
[27] A. Benatia *et al.*, "BestSF: A Sparse Meta-Format for Optimizing SpMV on GPU," *ACM TACO* 15(3), 2018.
[28] M. Burtscher, R. Nasre, K. Pingali, "A Quantitative Study of Irregular Programs on GPUs," *IISWC*, 2012.
[29] D. Merrill, M. Garland, A. Grimshaw, "Scalable GPU Graph Traversal," *PPoPP*, 2012.
[30] B. Hagedorn *et al.*, "Graphene: An IR for Optimized Tensor Computations on GPUs," *ASPLOS*, 2023.
[31] G. Diamos *et al.*, "Control Flow Emulation on Tiled SIMD Architectures," *CC*, 2008.
[32] N. Ding, S. Williams, "An Instruction Roofline Model for GPUs," *Concurrency Computat. Pract. Exper.*, 2022.
[33] S. Williams, A. Waterman, D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *CACM* 52(4), 2009.
[34] J. Ringlein *et al.*, "GPU Performance Portability needs Autotuning," arXiv:2505.03780, 2025.
[35] T. Chen *et al.*, "TVM: An Automated End-to-End Optimizing Compiler for Deep Learning," *OSDI*, 2018.